

SnapSight — Complete Project Build Guide

Qualcomm Snapdragon AI Lab Build & Present Challenge 2026

Recommended concept: A privacy-first multimodal AI assistant designed for Snapdragon-powered HP PCs.

Goal: Exact roadmap — kya banana hai, AI models kaise use karne hain, Snapdragon/NPU angle kaise prove karna hai, coding prompts kya dene hain, aur 10-day sprint mein project kaise complete karna hai.

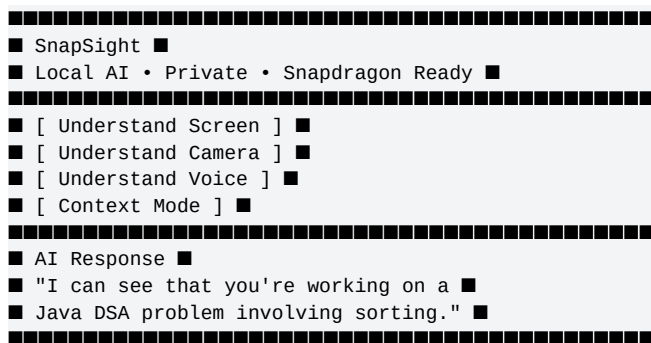
Important: Qualcomm AI Hub/model availability, target-device support aur runtimes change ho sakte hain. Exact model/runtime implementation ke time current official Qualcomm documentation se verify karo. NPU use aur benchmark numbers sirf actual evidence ke basis par claim karo.

1. SnapSight kya hai?

One-line: SnapSight is a privacy-first multimodal AI assistant that understands the user's screen, voice and visual context locally on a Snapdragon-powered HP PC.

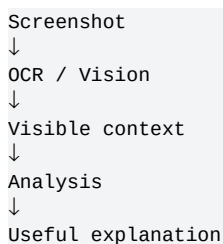


2. User interface



Pehle buttons independently working banao; Context Mode ko final integration feature rakho.

3. Understand Screen



Example: user Java code/DSA question kholta hai aur app visible context explain karta hai. MVP mein one screenshot → one inference → one response se start karo.

- Local/on-device processing is central to the architecture.
- Network dependency can be reduced for supported local workloads.
- Sensitive screen/audio/camera data can remain on-device by design.
- Snapdragon gives a concrete target for AI inference optimization and measurement.

9. AI models — start small

Speech

Whisper / Distil-Whisper family ka Snapdragon-compatible variant investigate karo. Exact model, target aur runtime current AI Hub documentation se verify karo.

Vision

Exact screen/camera task ke liye current Qualcomm AI Hub-compatible vision model choose karo. Target Snapdragon + Windows support ko selection ka primary constraint rakho.

OCR

Optional. Screen understanding ke liye needed ho to add karo, but first screen MVP complete karo.

Golden rule: Model, runtime, NPU support aur benchmarks guess nahi karne hain.

10. Folder structure

```

snap-sight/
├── frontend/
│   ├── index.html
│   ├── style.css
│   └── app.js
├── backend/
│   ├── main.py
│   ├── speech.py
│   ├── vision.py
│   ├── screen.py
│   ├── context.py
│   └── utils.py
├── models/
│   └── README.md
├── tests/
├── assets/
├── requirements.txt
└── README.md

```

11. Tech stack

- Frontend: HTML/CSS/JavaScript; React only if already comfortable.
- Backend: Python + FastAPI.
- Computer vision: OpenCV + selected model pipeline.
- Speech: Whisper/Distil-Whisper compatible model.
- Deployment: exact Qualcomm-supported workflow for the chosen model.
- Storage: JSON/SQLite only if needed.

12. Development order

```

MVP v0.1: SCREEN → AI → ANSWER

MVP v0.2: MIC → WHISPER → QUESTION

MVP v0.3: CAMERA → VISION → CONTEXT

MVP v1: SCREEN + MIC + CAMERA

```

```
↓  
CONTEXT ENGINE  
↓  
ANSWER
```

Har module ko run, test aur Git commit karke next module par jao.

13. Screen implementation

```
capture_screen()  
↓  
preprocess  
↓  
OCR / vision inference  
↓  
screen_context  
↓  
response
```

Test: known Java/DSA screen → button → relevant visible context.

14. Voice implementation

```
record_audio()  
↓  
preprocess  
↓  
Whisper inference  
↓  
transcript  
↓  
voice_query
```

Test: 5-10 second clear question → transcript visible in UI.

15. Camera implementation

```
capture_frame()  
↓  
preprocess  
↓  
vision inference  
↓  
camera_context  
↓  
result
```

Single-frame inference first. Continuous mode only if time remains.

16. Context Engine

```
context = {  
  "screen": screen_context,  
  "camera": camera_context,  
  "voice": user_question  
}  
  
final_response = context_engine(context)
```

Start with simple context combination. Optional small local language model baad mein add karo.

17. LLM kab add karna hai?

Large LLM se start mat karo. Screen + Voice + Camera pipelines working hone ke baad hi natural-language response generation ke liye small local model consider karo. Isse project manageable rahega.

18. Qualcomm AI Hub workflow

```
Find model
↓
Verify Snapdragon target
↓
Verify Windows/runtime
↓
Check input/output
↓
Evaluate where supported
↓
Collect actual metrics
↓
Integrate
↓
Document
```

Exact model/runtime support ko current AI Hub page par verify karo.

19. Performance evidence

```
Target device: [exact target]
OS: Windows 11
Model: [exact model]
Runtime: [verified runtime]
Compute: [NPU/CPU/GPU only if observed]
Inference: [measured]
Memory: [measured]
Input/test conditions: [actual]
```

Screenshots/logs preserve karo. '10x faster' tabhi likho jab controlled measurement ho.

20. AI coding workflow

```
Architecture
↓
One module
↓
Run locally
↓
Paste complete error
↓
Minimal fix
↓
Test
↓
Git commit
↓
Next module
```

One-shot 'make the whole app' prompt avoid karo.

21. Master prompt

I am building SnapSight for the Qualcomm Snapdragon AI Lab Build & Present Challenge 2026.

SnapSight is a privacy-first multimodal AI assistant designed for Snapdragon-powered HP PCs.

MVP:

1. Screen understanding
2. Camera understanding
3. Voice understanding
4. Context Engine

Tech:

Python, FastAPI, HTML/CSS/JavaScript, OpenCV, Qualcomm AI Hub compatible models, Windows.

Constraints:

- Do not invent Qualcomm APIs.

- Do not assume NPU execution without verification.
- Do not use fake benchmark numbers.
- Separate model inference from application logic.
- Modular architecture.
- Buildable within 10 days.

First DO NOT write code.

Give:

1. architecture
2. data flow
3. folder structure
4. dependencies
5. models to investigate
6. Qualcomm AI Hub integration points
7. development order
8. testing strategy
9. Snapdragon performance validation strategy

Explain in simple Hinglish.

22. Module coding prompt

Now implement module 1 only.

Module: Screen capture + screen understanding.

Requirements:

- Python
- FastAPI backend
- Clean error handling
- Separate model layer from API layer
- No hard-coded paths
- Logging
- Explain every file
- Windows setup commands
- Use only verified Qualcomm-compatible workflow.

Do not implement other modules.

At the end provide:

1. files
2. code
3. installation
4. test command
5. expected output
6. limitations

23. Debugging prompt

I am building SnapSight for Windows on Snapdragon.

Error:

[PASTE COMPLETE ERROR]

Environment:

Python:

OS:

Architecture:

Model:

Runtime:

Do not rewrite the entire project.

First:

1. root cause
2. incorrect line/file
3. why it fails
4. smallest fix
5. corrected snippet
6. verification command

Do not invent APIs or functions not present in the installed version.

24. Qualcomm verification prompt

Act as a Qualcomm on-device AI deployment engineer.

I want to deploy [MODEL] on Snapdragon [TARGET] Windows 11.

Before writing code:

1. verify model listing for target
2. identify supported runtime
3. verify NPU execution support
4. identify input/output format
5. identify correct AI Hub workflow
6. identify hosted Snapdragon validation path

Do not assume an API exists. If something cannot be verified, say so explicitly.

25. Context prompt

SCREEN CONTEXT:
[screen result]

CAMERA CONTEXT:
[camera result]

VOICE QUESTION:
[user question]

Generate a concise useful answer using only the available context. Clearly state when context is insufficient. Do not invent facts.

26. 10-day execution plan

Day	Task	Deliverable
1	Architecture + model/runtime research	Repo + verified candidates
2	Screen pipeline	Screenshot → AI result
3	Voice pipeline	Audio → transcript
4	Camera pipeline	Frame → vision result
5	Context Engine	Combine modalities
6	UI integration	Polished dashboard
7	Snapdragon validation	Target/runtime/compute evidence
8	Polish	Errors + UX
9	Docs + demo	README + screenshots + video
10	Final submission	Full test + clean repo + submit

27. Demo video — 2-3 minutes

- 0:00-0:20 — Problem: local context understanding without making the experience depend on the cloud.
- 0:20-0:40 — Introduce SnapSight.
- 0:40-1:10 — Understand Screen live demo.
- 1:10-1:35 — Understand Voice.

- 1:35-1:55 — Camera understanding.
- 1:55-2:20 — Context Mode: screen + voice question.
- 2:20-2:45 — Snapdragon/AI Hub target, runtime and actual performance evidence.
- End — README/repository and one-line value proposition.

28. Presentation story

SLIDE 1: AI should not always need the cloud.

SLIDE 2: Problem

Screen + Camera + Mic → Cloud

→ network dependency / privacy concerns

SLIDE 3: SnapSight

Screen + Camera + Mic → Snapdragon PC

→ Local AI → Context

SLIDE 4: Live Demo

SLIDE 5: Models + Qualcomm AI Hub

SLIDE 6: Actual performance evidence

SLIDE 7: Why on-device AI?

SLIDE 8: Future scope

29. README structure

```
# SnapSight
## Problem
## Solution
## Key Features
## Architecture
## AI Models
## Qualcomm AI Hub Integration
## Snapdragon Optimization
## Compute / NPU Evidence
## Installation
## Usage
## Performance
## Screenshots
## Demo
## Limitations
## Future Improvements
## License
```

30. What NOT to do

- 20-feature giant project mat banao.
- Scratch se LLM train mat karo.
- Unverified Qualcomm APIs copy-paste mat karo.
- Fake NPU/latency/FPS numbers mat likho.
- Sirf UI ko project mat samjho.
- Cloud API ko on-device AI mat claim karo.
- Continuous video AI se start mat karo.
- API keys/secrets repository mein upload mat karo.

31. Final technical checklist

- Screen module works.
- Voice module works.
- Camera module works.

- Context Mode combines at least two modalities reliably.
- Model target support verified.
- Runtime documented.
- NPU/compute evidence real.
- Benchmark conditions documented.
- No fake performance claims.
- Windows setup reproducible.
- README complete.
- Architecture diagram included.
- Demo video recorded.
- Screenshots ready.
- No secrets in repo.
- Final challenge form double-checked.

32. Official resources

Qualcomm AI Hub: <https://aihub.qualcomm.com/>

AI Hub Get Started: <https://aihub.qualcomm.com/get-started>

AI Hub Compute: <https://aihub.qualcomm.com/compute>

Windows on Snapdragon AI:

<https://www.qualcomm.com/developer/windows-on-snapdragon/windows-on-snapdragon-ai>

Snapdragon AI: <https://www.qualcomm.com/snapdragon/laptops-and-tablets/consumer/ai>

Challenge:

<https://unstop.com/competitions/crp-snapdragon-ai-lab-build-present-challenge-qualcomm-1748893>

33. Final blueprint

SNAP-SIGHT

SCREEN CAMERA MIC

■ ■ ■

↓ ↓ ↓

Vision Vision Whisper

■ ■ ■

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □



Context Engine



Local AI Response



UI

Snapdragon target



Verified runtime



NPU where verified



Measured evidence

FINAL DEMO:

"Ask your laptop what it sees,
what you're working on,
and what you just said –
without making the whole experience
depend on the cloud."

Execution principle: Pehle one feature end-to-end working banao. Then second, then integration.
Snapdragon evidence ko early research karo, but final performance claims only after testing.